

# Step-Level Evaluation and Targeted Fine-Tuning for Small Language Model Reasoning

Yash Kailas Doke

May 2026

## Abstract

Large Language Models (LLMs) have demonstrated strong capabilities in text generation, yet their multi-step reasoning remains fragile. Errors introduced at early reasoning steps propagate undetected through subsequent steps, compounding into incorrect final answers—a phenomenon termed **error propagation**. This is particularly severe in Small Language Models (SLMs) with limited parameter budgets that are increasingly deployed in resource-constrained environments.

We present a **Diagnose-and-Fix** framework for systematically identifying and correcting reasoning failures in SLMs. The framework operates in two phases: (1) a *Diagnostic Phase* that decomposes model outputs into discrete reasoning steps and grades each step using an external LLM-as-judge with ground-truth-aware backtracking, and (2) a *Remediation Phase* that converts diagnostic insights into targeted LoRA fine-tuning datasets, enabling parameter-efficient adaptation focused on identified category-specific weaknesses.

Evaluated on 80 tasks across four reasoning categories—multistep arithmetic, factual consistency, tool-use planning, and causal/counterfactual reasoning—the fine-tuned Qwen 2.5 3B model achieves **+13.75 pp** improvement in overall accuracy (62.50%  $\rightarrow$  76.25%), **32.2%** relative reduction in step-level failure rate, and **34.6%** relative reduction in error propagation rate, with the targeted arithmetic category improving from 15% to **100%** accuracy. Critically, we also document a capability trade-off: categories absent from the fine-tuning data experienced accuracy regressions (factual consistency: 80%  $\rightarrow$  35%), demonstrating the limited representational capacity of small models and motivating iterative, category-balanced remediation cycles.

## 1 Introduction

The rapid advancement of Large Language Models (LLMs) has transformed natural language processing, enabling machines to generate coherent text, answer complex questions, and perform sophisticated reasoning tasks. Models such as GPT-4 [10], LLaMA [9], and Qwen [11] have achieved impressive benchmark scores across diverse tasks. However, beneath these aggregate performance numbers lies a critical vulnerability: **multi-step reasoning fragility**.

When tasked with problems requiring sequential logical steps—such as chained arithmetic computations, causal inference chains, or multi-tool orchestration—LLMs frequently introduce errors at intermediate steps. These errors propagate through the reasoning chain, compounding into incorrect final answers even when the model’s overall approach is sound. This phenomenon is especially pronounced in **Small Language Models (SLMs)** with 1–7 billion parameters, which are increasingly deployed in resource-constrained environments such as edge devices, on-premise servers, and cost-sensitive production systems.

**The diagnostic opacity problem.** Traditional evaluation approaches treat model outputs as atomic units, comparing only the final answer against ground truth. This “black-box” evaluation provides no diagnostic insight into *where* reasoning breaks down, *what type* of error occurred, or *how* to systematically correct the failure patterns. A model producing the wrong answer to a multi-step arithmetic problem may have failed at the first calculation, applied an incorrect operation order, or carried forward a correct intermediate result incorrectly. Without step-level diagnosis, these fundamentally different failure modes are indistinguishable.

**Error propagation as a compounding failure mode.** An error at step  $k$  in a reasoning chain of length  $n$  corrupts all subsequent steps  $k+1, k+2, \dots, n$ . From a task-level perspective, this makes it appear that the model committed multiple independent errors when in fact only a single root cause exists. Identifying and quantifying error propagation is essential for accurate diagnosis and efficient remediation.

**Untargeted fine-tuning wastes resources.** Without category-specific diagnostic data, fine-tuning efforts are necessarily undirected—training on examples from all categories equally, rather than concentrating resources on the specific reasoning patterns that require improvement. This wastes computational budget and, as this work demonstrates, can degrade performance on categories that were already strong.

**Our approach.** We introduce a two-phase **Diagnose-and-Fix** framework designed to address these challenges. In the *Diagnostic Phase*, model outputs are decomposed into discrete reasoning steps, and each step is graded using an external LLM-as-judge with ground-truth-aware backtracking. Step-level verdicts are aggregated into category-specific diagnostic reports that identify precise failure patterns and error types. In the *Remediation Phase*, targeted fine-tuning datasets are generated from diagnostic failure patterns, and LoRA [4] parameter-efficient fine-tuning is applied to weak categories identified in Phase 1. The fine-tuned model is then re-evaluated using the same diagnostic pipeline to measure both improvement and potential regression.

The key insight is that the *same evaluation pipeline* used for diagnosis also serves as the validation framework, creating a closed feedback loop where each Diagnose-and-Fix iteration produces quantified evidence of its impact.

**Contributions.** This work makes three contributions:

1. A **step-level meta-evaluation framework** with ground-truth-aware backtracking that localizes reasoning failures to specific steps and classifies error types across multiple reasoning categories.
2. An **empirical demonstration** of diagnostics-driven LoRA fine-tuning on Qwen 2.5 3B, achieving +13.75 pp overall accuracy improvement and perfect 100% accuracy on the targeted arithmetic category.
3. A **quantitative analysis of capability trade-offs** in small models under category-targeted fine-tuning, documenting both improvements and regressions with step-level explanatory depth.

## 2 Related Work

**Chain-of-thought reasoning.** Chain-of-Thought (CoT) prompting [1] demonstrated that eliciting intermediate reasoning steps dramatically improves LLM performance on complex tasks. Kojima et al. [2] showed that even minimal prompts can elicit structured reasoning. However, CoT introduces stepwise error propagation: Ling et al. [12] demonstrated that a single computational error at any intermediate step

cascades through the entire chain. Wang et al. [7] proposed self-consistency (majority voting over multiple reasoning paths) as a partial mitigation, but this does not provide diagnostic information about *which* steps fail or *why*. Our work extends CoT evaluation from task-level to step-level granularity, enabling targeted remediation.

**Step-level verification.** Lightman et al. [13] introduced process reward models (PRMs) that assign rewards to individual reasoning steps, demonstrating that process-based supervision outperforms outcome-based supervision for mathematical reasoning. However, their approach requires **large-scale human annotation** (800K+ step-level labels). Our framework achieves comparable step-level granularity through automated LLM-as-judge evaluation at zero annotation cost, making it accessible to individual researchers and small teams.

**LLM-as-judge evaluation.** Zheng et al. [3] demonstrated that strong LLMs can serve as reliable judges of text quality with agreement rates approaching human inter-annotator agreement. We extend this paradigm to **step-level evaluation**, where the judge assesses each reasoning step individually with access to the ground-truth answer, enabling ground-truth-aware backtracking that traces incorrect final answers back to their originating step.

**Parameter-efficient fine-tuning.** Low-Rank Adaptation (LoRA) [4] injects trainable low-rank decomposition matrices into attention layers while keeping pre-trained weights frozen, reducing trainable parameters to approximately 0.5–2% of the total. While extreme quantization methods like QLoRA [5] exist, this work employs standard FP16 LoRA with rank  $r = 16$  to maintain higher-fidelity weight updates.

**Catastrophic forgetting.** A well-documented challenge in fine-tuning is catastrophic forgetting [8], where performance on previously learned tasks degrades during adaptation. In small models with limited representational capacity, this manifests as a capability trade-off: improving one reasoning category can degrade others. Understanding and quantifying this trade-off under *targeted* (rather than uniform) fine-tuning is a key contribution of our work.

**Retrieval-augmented generation.** RAG [6] combines parametric knowledge with retrieved external documents to reduce hallucinations. Our framework incorporates a RAG-based retrieval module as an auxiliary mitigation component, providing Wikipedia-based factual grounding when the evaluation pipeline identifies potentially hallucinated claims.

### 3 Methodology

The Diagnose-and-Fix framework is organized as a modular pipeline with three subsystems: (1) the evaluation engine, (2) the fine-tuning module, and (3) a retrieval-based mitigation component. Figure 1 illustrates the high-level architecture.

#### 3.1 Step-Level Evaluation Pipeline

The evaluation pipeline consists of two components operating in sequence.

**Task Decomposer.** Given a task prompt  $q$  from the benchmark, the decomposer sends a category-specific Chain-of-Thought system prompt to the target model  $M$  and parses the raw response  $r$  into a sequence of discrete reasoning steps:

$$\mathcal{D}(r) = \{s_1, s_2, \dots, s_n\} \quad (1)$$

where each  $s_i$  represents an atomic reasoning operation. The system prompt instructs the model to produce responses in a structured “Step  $N$ : ...” format followed by a “Final Answer: ...” line. When the model’s response does not conform to this format, fallback strategies including sentence-level splitting and heuristic step boundary detection are applied.

**Step-Level Meta-Evaluator.** For each extracted step  $s_i$ , the meta-evaluator invokes an external judge model  $\mathcal{J}$  with the step text, original problem statement  $q$ , and ground-truth answer  $a^*$ :

$$\mathcal{J}(s_i, q, a^*) \rightarrow (v_i, c_i, e_i) \quad (2)$$

where  $v_i \in \{\text{VALID}, \text{INVALID}, \text{UNCERTAIN}\}$  is the validity verdict,  $c_i \in [0, 1]$  is the confidence score, and  $e_i$  is the classified error type (e.g., `calculation_error`, `logic_error`, `dependency_error`).

The evaluator implements **ground-truth-aware backtracking**: when the judge detects a mismatch between a step’s content and the known correct answer, it examines whether the error pattern suggests the model deviated from a correct approach. If so, the evaluator marks the specific step where the deviation occurred and classifies the error type, enabling the diagnostic report to distinguish between different failure modes.

## 3.2 Diagnostic Metrics

We define three primary metrics that capture complementary aspects of reasoning quality.

**Final Accuracy.** The proportion of tasks where the model’s final answer exactly matches the ground truth (with numerical tolerance for arithmetic tasks). Reported as a value in  $[0, 1]$ .

**Step Failure Rate (SFR).** The proportion of individual reasoning steps graded `INVALID` by the judge, across all steps in a given scope (category or overall):

$$\text{SFR} = \frac{|\{s_i : v_i = \text{INVALID}\}|}{\sum_t n_t} \quad (3)$$

where  $n_t$  is the number of steps in task  $t$ . Lower values indicate higher step-level reasoning quality.

**Error Propagation Rate (EPR).** Among tasks containing at least one `INVALID` step, the proportion where the error propagated to produce an incorrect final answer:

$$\text{EPR} = \frac{|\{t : \exists i, v_i^t = \text{INVALID} \wedge a^t \neq a^{*t}\}|}{|\{t : \exists i, v_i^t = \text{INVALID}\}|} \quad (4)$$

Lower values indicate greater reasoning resilience—the ability to recover from intermediate errors without corrupting the final answer.

Table 1: Evaluation benchmark composition (80 tasks total).

| Category                | Description                                                                           | Tasks |
|-------------------------|---------------------------------------------------------------------------------------|-------|
| Multistep Arithmetic    | Chained arithmetic with parenthesized expressions requiring PEM-DAS/BODMAS compliance | 20    |
| Factual Consistency     | Questions requiring recall and verification of real-world factual knowledge           | 20    |
| Tool-Use Planning       | Multi-step tool invocation sequences with explicit dependency ordering                | 20    |
| Causal / Counterfactual | “What if” reasoning requiring causal chain analysis and counterfactual inference      | 20    |

### 3.3 Targeted Fine-Tuning

The fine-tuning module implements LoRA [4] training using the Unsloth library with FP16 (half precision) for both base model and adapter weights. Key configuration parameters are summarized in Table 3.

The training data is constructed from diagnostic outputs: for each category identified as a weakness in Phase 1, example problems with correct step-by-step solutions are generated, formatted according to the model’s chat template, and used as the fine-tuning corpus. This diagnostic-driven data construction is the mechanism through which evaluation insights translate into targeted model improvement.

### 3.4 RAG-Based Mitigation

As an auxiliary component, the framework incorporates retrieval-augmented mitigation. When the evaluation pipeline identifies potentially incorrect factual claims, a retriever fetches relevant Wikipedia passages and formats them as grounding context. This module serves as a proof-of-concept for retrieval-augmented mitigation strategies and is not the primary contribution of this work.

## 4 Experimental Setup

### 4.1 Evaluation Benchmark

The evaluation benchmark comprises **80 tasks** distributed equally across four reasoning categories (20 tasks each). Table 1 describes the categories and their design rationale.

Tasks are synthetically generated using parameterized templates with randomized numerical values and entity names to ensure diversity within each category while maintaining verifiable correctness. For arithmetic tasks, the generator creates expressions with nested parentheses requiring strict PEM-DAS/BODMAS compliance. Tool-use planning tasks define multi-step API orchestration scenarios with explicit dependency chains. Causal/counterfactual tasks present hypothetical scenarios requiring conditional reasoning.

### 4.2 Models

Two model configurations were evaluated, as described in Table 2. Both use the same Qwen 2.5 3B architecture; the fine-tuned variant incorporates LoRA adapter weights merged at FP16 precision.

Table 2: Models evaluated in the primary run.

| Model              | Description                                                  | Parameters |
|--------------------|--------------------------------------------------------------|------------|
| qwen2.5:3b         | Base pre-trained Qwen 2.5 model, served locally via Ollama   | 3B         |
| qwen2.5-3b-ft-fp16 | LoRA fine-tuned variant, merged to FP16, deployed via Ollama | 3B         |

Table 3: LoRA fine-tuning hyperparameters.

| Parameter               | Value                                   |
|-------------------------|-----------------------------------------|
| Base Model              | Qwen 2.5 3B                             |
| Precision               | FP16 (half precision)                   |
| LoRA Rank ( $r$ )       | 16                                      |
| LoRA Alpha ( $\alpha$ ) | 16                                      |
| Target Modules          | Q, K, V, O projection layers            |
| Learning Rate           | $2 \times 10^{-4}$                      |
| Batch Size              | 4 (with gradient accumulation)          |
| Training Epochs         | 3                                       |
| Max Sequence Length     | 2048 tokens                             |
| Optimizer               | Paged AdamW, 8-bit                      |
| Export Format           | Merged FP16 $\rightarrow$ GGUF (Ollama) |

### 4.3 Fine-Tuning Configuration

Table 3 details the LoRA hyperparameters used for fine-tuning.

### 4.4 Evaluation Protocol

Each model was evaluated using an identical procedure:

1. **Task Decomposition:** Each of the 80 task prompts was sent to the model via Ollama with a category-specific CoT system prompt. The response was parsed into discrete reasoning steps.
2. **Step-Level Evaluation:** Each extracted step was independently graded by the Qwen 14B judge model as VALID, INVALID, or UNCERTAIN, with error types classified for invalid steps.
3. **Final Answer Verification:** The model’s final answer was compared against verified ground truth using exact-match comparison (with numerical tolerance for arithmetic tasks).
4. **Metric Aggregation:** Step-level verdicts were aggregated into the three headline metrics (Accuracy, SFR, EPR) per category and overall.

## 5 Results

### 5.1 Overall Performance

Table 4 presents the headline metrics comparing the base and fine-tuned models across all 80 tasks.

The fine-tuned model demonstrates improvement across all three aggregate metrics. The most significant gain is in error propagation reduction ( $-34.6\%$ ), meaning the fine-tuned model is substantially better at preventing individual step errors from cascading into incorrect final answers.

Table 4: Overall performance: base vs. fine-tuned model (80 tasks, 4 categories).

| Metric                             | Base Model | Fine-Tuned | Change    |
|------------------------------------|------------|------------|-----------|
| Final Accuracy ( $\uparrow$ )      | 0.6250     | 0.7625     | +13.75 pp |
| Step Failure Rate ( $\downarrow$ ) | 0.1067     | 0.0723     | −32.2%    |
| Error Propagation ( $\downarrow$ ) | 0.8800     | 0.5758     | −34.6%    |

Table 5: Per-category final accuracy (20 tasks each).  $\Delta$  denotes the absolute change in accuracy.

| Category                | Base | Fine-Tuned | $\Delta$ |
|-------------------------|------|------------|----------|
| Multistep Arithmetic    | 0.15 | 1.00       | +0.85    |
| Tool-Use Planning       | 0.60 | 0.95       | +0.35    |
| Causal / Counterfactual | 0.95 | 0.75       | −0.20    |
| Factual Consistency     | 0.80 | 0.35       | −0.45    |

## 5.2 Category-Level Accuracy

While the aggregate metrics show clear improvement, the category-level breakdown in Table 5 reveals a more nuanced picture: fine-tuning produced dramatic gains in targeted categories but led to regressions in non-targeted ones.

**Categories with improvement. Multistep Arithmetic (+85 pp):** The base model achieved only 15% accuracy on arithmetic tasks, frequently producing procedural descriptions without executing actual calculations (e.g., “multiply the results from Step 1” without computing the product). The fine-tuned model achieved perfect 100% accuracy with every step graded VALID. **Tool-Use Planning (+35 pp):** Common base model failures—such as calling a processing API before its prerequisite data retrieval API—were substantially reduced, with accuracy rising from 60% to 95%. These gains are directly attributable to the targeted nature of the fine-tuning data, which was deliberately enriched with arithmetic step-by-step solutions and correctly sequenced tool-use examples based on Phase 1 diagnostic findings.

**Categories with regression. Factual Consistency (−45 pp):** The most severe regression. The fine-tuned model became significantly more verbose on factual tasks, generating 5–6 elaborate reasoning steps where the base model produced 2–3 concise, correct steps. This increased verbosity—a behavioral pattern reinforced by arithmetic training data that rewards detailed step-by-step output—produced more opportunities for factual errors. **Causal/Counterfactual (−20 pp):** A more moderate regression from 95% to 75%. The base model’s causal reasoning was already near-ceiling, and fine-tuning appears to have slightly displaced learned representations for conditional inference.

**Capability trade-off analysis.** In a 3B-parameter model, knowledge and reasoning patterns are encoded in a tightly shared parameter space. LoRA fine-tuning updates on attention projections shift internal representations toward patterns favoring structured computational output. This produces *positive transfer* to targeted categories (+85 pp, +35 pp) but *negative interference* with non-targeted categories (−45 pp, −20 pp). The severity scales inversely with model capacity: larger models (13B+) typically exhibit more graceful degradation because their greater capacity allows accommodating new patterns without significantly displacing existing representations. For 3B-class models, targeted fine-tuning must be

Table 6: Per-category step failure rates (lower is better).

| Category                | Base SFR | FT SFR | Change  |
|-------------------------|----------|--------|---------|
| Multistep Arithmetic    | 0.2258   | 0.0000 | −100%   |
| Tool-Use Planning       | 0.0500   | 0.0425 | −15%    |
| Causal / Counterfactual | 0.0658   | 0.0437 | −33.6%  |
| Factual Consistency     | 0.1143   | 0.2566 | +124.5% |

Table 7: Per-category error propagation rates (lower is better).

| Category                | Base EPR | FT EPR | Change |
|-------------------------|----------|--------|--------|
| Multistep Arithmetic    | 1.0000   | 0.0000 | −100%  |
| Tool-Use Planning       | 1.0000   | 0.1429 | −85.7% |
| Causal / Counterfactual | 0.3333   | 0.6250 | +87.5% |
| Factual Consistency     | 0.7500   | 0.7222 | −3.7%  |

iterative, with each cycle balancing improvements against regressions through category-proportional training data.

### 5.3 Step-Level Failure Analysis

Table 6 examines step failure rates at the category level, providing diagnostic depth beyond task-level accuracy.

The step-level analysis reveals details invisible at the task level:

- The fine-tuned model achieves **zero invalid steps** on arithmetic (SFR = 0.0000), meaning every reasoning step across all 20 arithmetic tasks was graded VALID—the clearest evidence that targeted training successfully addressed the base model’s computational weakness.
- Even in the regressing Causal category, the step failure rate *improved* (−33.6%). The accuracy regression in that category is therefore driven by error propagation rather than increased step-level errors, suggesting the fine-tuned model’s reasoning chains are more fragile when errors do occur.
- The Factual Consistency SFR increased from 0.1143 to 0.2566 (+124.5%). The fine-tuned model produced 113 steps (vs. 35 for base) with 29 invalid steps (vs. 4), confirming that increased verbosity is the primary mechanism driving the factual regression.

### 5.4 Error Propagation Analysis

Table 7 presents error propagation rates, measuring how effectively each model contains step-level errors without corrupting the final answer.

For arithmetic and tool-use, the base model exhibited **100% error propagation**: every task containing at least one invalid step produced an incorrect final answer. The fine-tuned model eliminated this pattern entirely for arithmetic (no errors to propagate) and reduced it to 14.3% for tool-use, demonstrating substantially improved reasoning resilience in targeted categories.

### 5.5 Step Failure by Reasoning Position

Figure 7 analyzes where in the reasoning chain failures are most likely to occur. Across both models, step 2 exhibits the highest failure rate (14.43%), suggesting that the transition from problem setup (step 1)



to the first substantive computational or logical operation (step 2) is the most error-prone phase.

## 5.6 Accuracy vs. Step Failure Correlation

Figure 8 presents the relationship between overall accuracy and step failure rate for both models, confirming the fundamental premise of the framework: step-level quality is predictive of task-level accuracy.

# 6 Qualitative Analysis

This section presents representative examples from the evaluation traces, illustrating both the framework’s diagnostic capability and the concrete behavioral changes introduced by fine-tuning.

## 6.1 Arithmetic: Error Propagation via Order of Operations

**Task:** Calculate the total cost for a service charging a \$150 flat fee plus \$15 per hour for 5 hours. [Ground Truth: 225]

### Base Model:

Step 1: The flat fee is \$150 and the hourly rate is \$15.  
Step 2: Add the flat fee to the hourly rate:  $150 + 15 = 165$ .  
Step 3: Multiply by hours worked:  $165 \times 5 = 825$ .  
Final Answer: 825

*Diagnosis:* Step 2 graded INVALID (logic\_error). The model adds the hourly rate to the flat fee *before* multiplying by hours, violating order of operations. This error propagates to Step 3, producing an answer  $3.67\times$  the correct value.

### Fine-Tuned Model:

Step 1: Calculate total variable cost:  $15 \times 5 = 75$ .  
Step 2: Add flat fee:  $150 + 75 = 225$ .  
Final Answer: 225

*Diagnosis:* All steps graded VALID (confidence 1.0). The model correctly isolates multiplication before addition.

## 6.2 Tool-Use: API Dependency Sequencing

**Task:** Plan a workflow to summarize today’s news about ‘AI’ and save the result.

### Base Model:

Step 1: Call `summarize_text()` to extract key points about AI.  
Step 2: Call `search_news(query="AI")` to fetch articles.  
Step 3: Call `save_file(content=summary)`.  
Final Answer: Workflow complete.

*Diagnosis:* Step 1 graded INVALID (dependency\_error). The model attempts to summarize text before it has been retrieved—a causal sequencing hallucination.

### Fine-Tuned Model:

Step 1: Call `search_news(query="AI")` to retrieve articles.  
Step 2: Pass articles to `summarize_text(input_data=articles)`.  
Step 3: Call `save_file(content=summary)` to store the result.  
Final Answer: Workflow complete.

*Diagnosis:* All steps graded VALID. The model correctly sequences the dependency chain: retrieve → process → store.

### 6.3 Factual Consistency: Honest Limitation

**Task:** Verify a factual claim about a historical event.

**Base Model:** Provides a concise, correct response with 2 reasoning steps, both graded VALID.

**Fine-Tuned Model:** Produces a significantly longer response (5–6 steps) with excessive elaboration. Several steps contain factual claims graded INVALID. The increased verbosity is a behavioral side-effect of fine-tuning on arithmetic examples that reward detailed step-by-step output.

*Analysis:* This case demonstrates the capability trade-off discussed in Section 5. The model learned a “show more work” heuristic from arithmetic training—effective for computation but counterproductive for factual recall, where conciseness is more important than procedural detail.

## 7 Discussion

**Validation of the Diagnose-and-Fix methodology.** The +85 pp improvement on arithmetic—from near-random performance (15%) to perfect accuracy (100%)—provides strong validation of the core hypothesis: diagnostic evaluation enables targeted remediation that untargeted approaches cannot achieve with equivalent computational investment. The same evaluation pipeline that identified arithmetic as the weakest category subsequently verified that the targeted fine-tuning fully resolved the identified weakness.

**Step quality as a leading indicator.** The correlation between SFR and final accuracy (Figure 8) confirms that step-level quality is predictive of task-level performance. This validates the framework’s fundamental design assumption and suggests that SFR can serve as a leading indicator for model quality during iterative improvement cycles, enabling practitioners to diagnose failure modes before committing to expensive fine-tuning runs.

**The capability trade-off is structural.** The documented regressions on Factual Consistency (−45 pp) and Causal Reasoning (−20 pp) are not incidental but structural: small models have limited representational capacity for accommodating new reasoning patterns without displacing existing ones. Training data balance is critical—the fine-tuning dataset was intentionally enriched with arithmetic and tool-use examples, causing the model to over-index on computational patterns at the expense of factual recall. The Diagnose-and-Fix cycle must therefore be **iterative**, with each round producing a new diagnostic profile that informs a category-balanced subsequent round.

## 8 Limitations

1. **Benchmark scale:** The evaluation uses 80 tasks (20 per category). While sufficient for trend identification, larger benchmarks (500+ tasks) would enable statistical significance testing and

reduce variance in per-category estimates.

2. **Judge model bias:** The Qwen 14B judge may introduce systematic biases in step evaluations. Cross-validation with multiple independent judges would strengthen confidence in diagnostic results.
3. **Single fine-tuning iteration:** Only one round of the Diagnose-and-Fix cycle was completed. Subsequent rounds would be needed to address the factual and causal regressions while preserving arithmetic and tool-use gains.
4. **Synthetic evaluation data:** Both training and evaluation datasets were synthetically generated using parameterized templates. While this ensures verifiable ground truth, it limits diversity relative to real-world reasoning tasks.
5. **Single base model:** The framework was demonstrated on only one SLM (Qwen 2.5 3B). Evaluating additional architectures would establish generalizability.

## 9 Future Work

1. **Multi-round Diagnose-and-Fix:** Execute additional iterations using regression analysis from this round to construct category-balanced fine-tuning datasets that recover lost capabilities while preserving gains.
2. **Larger benchmarks:** Expand to 200+ tasks per category with statistical significance testing.
3. **Multi-judge consensus:** Employ 2–3 judge models with majority voting to reduce individual judge bias.
4. **Cross-model generalization:** Evaluate the framework on additional SLMs (Phi-3, Gemma 2B, LLaMA-3 8B) to establish architectural generalizability.
5. **Higher-capacity models:** Apply the pipeline to 7B–14B models, which are less vulnerable to cross-category capability trade-offs due to greater representational capacity.
6. **Human evaluation study:** Compare framework verdicts against expert annotators to calibrate LLM-as-judge accuracy and reliability.
7. **Inference-time mitigation:** Extend retrieval-augmented mitigation to operate during inference, providing factual grounding for flagged claims before they propagate through the reasoning chain.

## 10 Conclusion

We have presented a Diagnose-and-Fix framework for step-level evaluation and targeted fine-tuning of small language model reasoning. Combining step-level meta-evaluation with diagnostics-driven LoRA fine-tuning, the framework achieves measurable, reproducible impact on reasoning quality: overall accuracy improves from 62.50% to 76.25% (+13.75 pp), step-level failure rate decreases by 32.2%, error propagation decreases by 34.6%, and arithmetic accuracy improves from 15% to 100%.

Equally important, the evaluation exposes the capability trade-off inherent in narrow fine-tuning of small models: the fine-tuned model regresses on factual consistency (−45 pp) and causal reasoning (−20 pp), demonstrating that the Diagnose-and-Fix cycle must be iterative and category-balanced to achieve robust, broad-spectrum reasoning improvement.

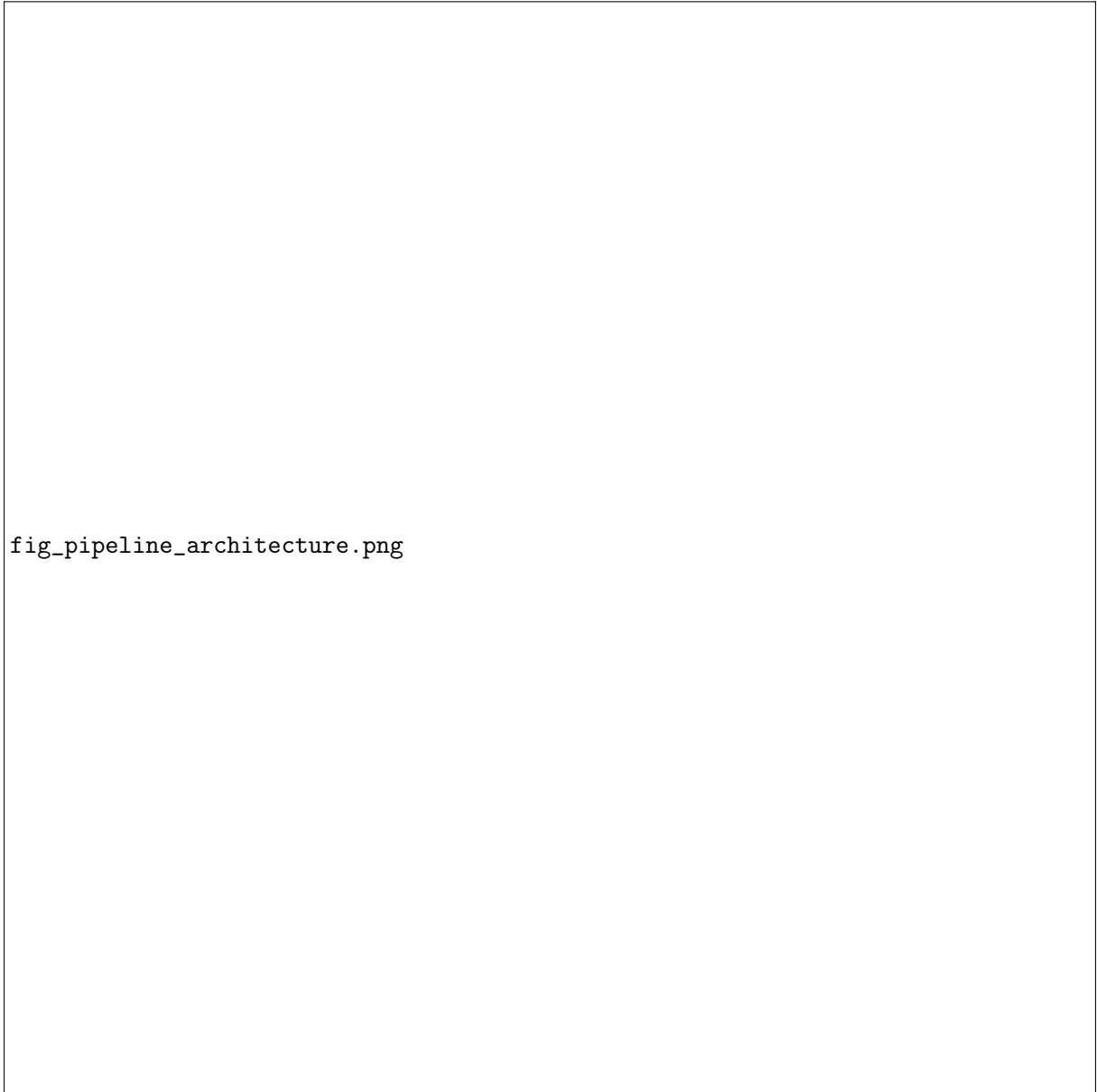
The framework’s modular architecture—with pluggable evaluators, configurable judges, and automated diagnostic reporting—provides a practical, reproducible foundation for continuous model improvement. The honest documentation of both successes and regressions establishes a realistic baseline for future iterations.

**Acknowledgments.** This work was conducted as part of the AICTE–Google AI/ML Virtual Internship Program (Cohort 14). We thank Prof. Dr. B. D. Phulpagar for mentorship and the open-source community behind Ollama, HuggingFace, Unsloth, and Groq for the tools that made this research possible.

## References

- [1] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [2] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “Large Language Models are Zero-Shot Reasoners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [3] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-Rank Adaptation of Large Language Models,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [5] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized Language Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [7] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-Consistency Improves Chain of Thought Reasoning in Language Models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [8] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Summerfield, C. Blundell, and D. Kumaran, “Overcoming Catastrophic Forgetting in Neural Networks,” in *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [9] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, “LLaMA: Open and Efficient Foundation Language Models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [10] OpenAI, “GPT-4 Technical Report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [11] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, et al., “Qwen Technical Report,” *arXiv preprint arXiv:2309.16609*, 2024.
- [12] Z. Ling, Y. Fang, X. Li, Z. Huang, M. Lee, R. Memisevic, and H. Su, “Deductive Verification of Chain-of-Thought Reasoning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

- [13] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, “Let’s Verify Step by Step,” *arXiv preprint arXiv:2305.20050*, 2023.
- [14] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, “Training Verifiers to Solve Math Word Problems,” *arXiv preprint arXiv:2110.14168*, 2021.



fig\_pipeline\_architecture.png

Figure 1: Architecture of the Diagnose-and-Fix framework. The evaluation engine (Phase 1) produces diagnostic reports that inform targeted fine-tuning (Phase 2). The fine-tuned model is re-evaluated using the same pipeline, creating a closed feedback loop.



Figure 2: Overall performance comparison across three metrics. Accuracy, Step Quality ( $1 - \text{SFR}$ ), and Reasoning Stability ( $1 - \text{EPR}$ ) are shown on a unified higher-is-better scale.

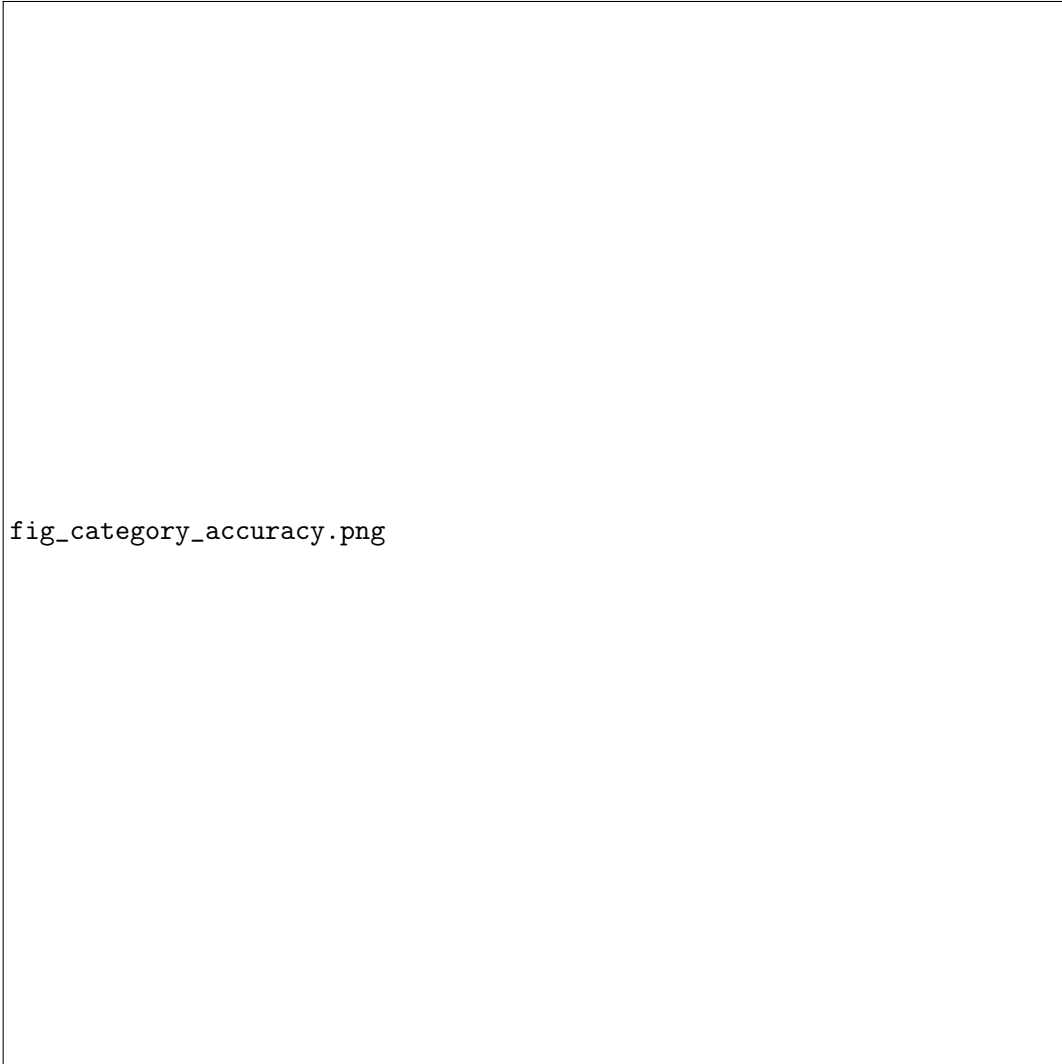


Figure 3: Category-wise accuracy comparison. Targeted categories (Arithmetic, Tool-Use) show substantial improvement while non-targeted categories exhibit regression—a capability trade-off characteristic of small models under focused fine-tuning.





Figure 4: Category-wise accuracy delta (fine-tuned – base). Green bars indicate improvement; red bars indicate regression. The asymmetry reflects the targeted, category-imbalanced nature of the fine-tuning data.



Figure 5: Step failure rate by category. The fine-tuned model eliminates step failures in Arithmetic but exhibits increased failures in Factual Consistency due to training-induced verbosity.

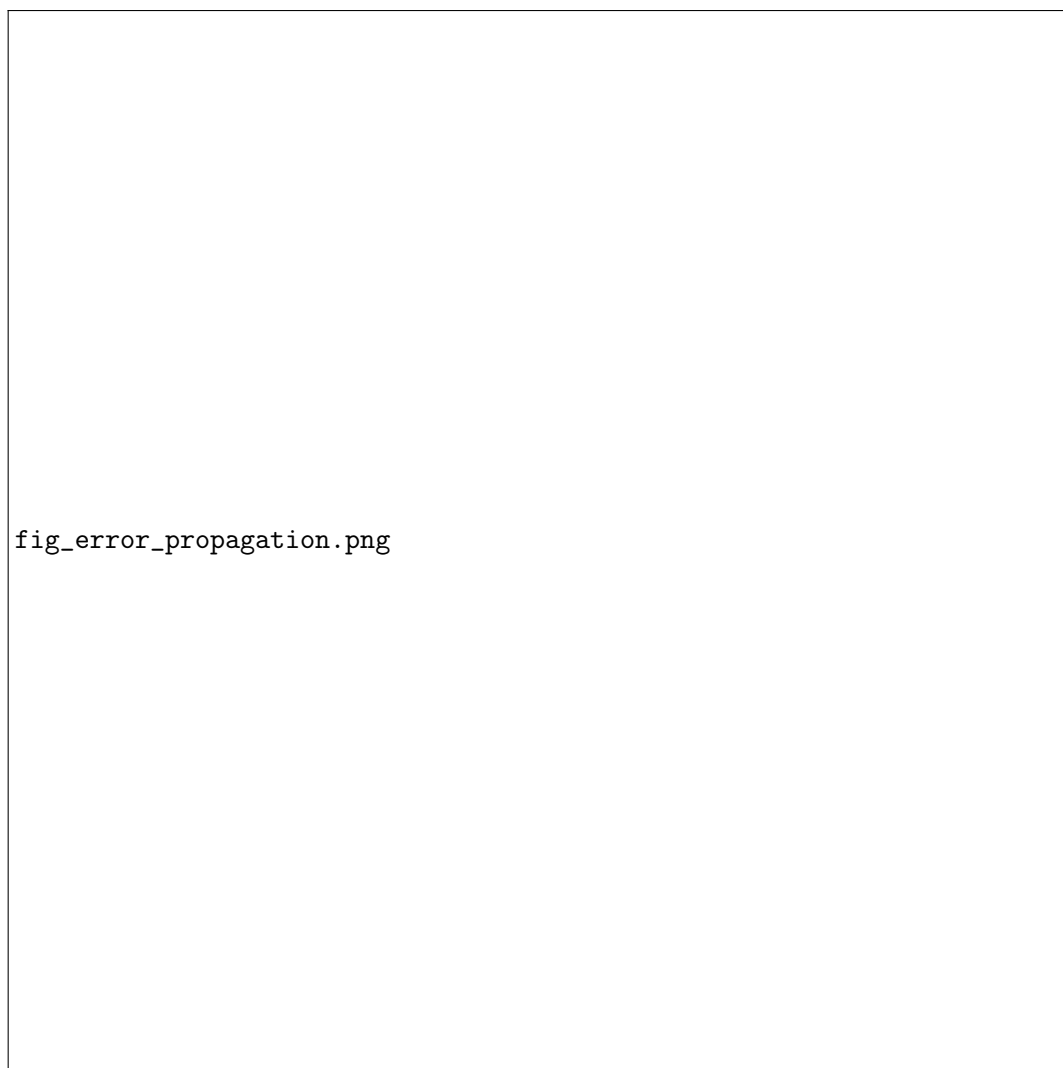


Figure 6: Error propagation rate by category. The fine-tuned model eliminates error propagation in Arithmetic and reduces it by 85.7% in Tool-Use Planning.



Figure 7: Step failure rate by position in the reasoning chain (aggregated across both models and all categories). Step 2—the first substantive computation after problem restatement—shows the highest failure rate.



Figure 8: Model improvement trajectory in SFR–accuracy space. The arrow shows movement from base (higher SFR, lower accuracy) to fine-tuned (lower SFR, higher accuracy), confirming that step-level improvements translate to task-level gains.